# Advanced Database Systems Portfolio

**MIT 8103 - Advanced Database Systems**

**Student Name:** Oluwatosin Ogunjemilua
**Student ID:** 301904697

**Matriculation Number:** 2026/A/MIT/0050
**Academic Session:** 2026/2027
**Course:** MIT 8103 Advanced Database Systems

**Git Hub Repository url:** https://github.com/TosinKyle/MIT8103-Advanced-Database-Systems-Apex-Digital-Bank

# Portfolio 2: Query Processing and Optimization

## ➢ Database Creation

The database was implemented in Microsoft SQL Server based on the ERD developed in Portfolio 1. Database Implementation Query below:

==========

**CREATE DATABASE**

==========

```
CREATE DATABASE ApexDigitalBank;
GO
USE ApexDigitalBank;
GO
```

**BRANCH**

```
CREATE TABLE Branch (
    BranchID INT PRIMARY KEY,
    BranchName VARCHAR(100) NOT NULL,
    BranchCode VARCHAR(10) UNIQUE NOT NULL,
    StreetAddress VARCHAR(150),
    City VARCHAR(50),
    State VARCHAR(50),
    Country VARCHAR(50),
    PhoneNumber VARCHAR(15)
```

```
);
```

**CUSTOMER**

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    DateOfBirth DATE,
    Gender CHAR(1),
    PhoneNumber VARCHAR(15),
    Email VARCHAR(50),
    StreetAddress VARCHAR(150),
    City VARCHAR(50),
    State VARCHAR(50),
    Country VARCHAR(50),
    PostalCode VARCHAR(20),
    CustomerType VARCHAR(20),
    RegistrationDate DATETIME,
    Status VARCHAR(20)
);
 KYC RECORD
CREATE TABLE KYC_Record (
    KYC_ID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    BVN VARCHAR(11) UNIQUE,
    DocumentType VARCHAR(50),
    DocumentNumber VARCHAR(50),
    VerificationDate DATETIME,
    VerificationStatus VARCHAR(20),

    CONSTRAINT FK_KYC_Customer
    FOREIGN KEY (CustomerID)
    REFERENCES Customer(CustomerID)
```

```
);
```

**ACCOUNT**

```
CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    BranchID INT NOT NULL,
    AccountNumber VARCHAR(20) UNIQUE NOT NULL,
    AccountType VARCHAR(20),
    Balance DECIMAL(18,2),
    Currency VARCHAR(10),
    DateOpened DATETIME,
    AccountStatus VARCHAR(20),

    CONSTRAINT FK_Account_Customer
    FOREIGN KEY (CustomerID)
    REFERENCES Customer(CustomerID),

    CONSTRAINT FK_Account_Branch
    FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID)
);
```

**LOAN**

```
CREATE TABLE Loan (
    LoanID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    LoanType VARCHAR(30),
    LoanAmount DECIMAL(18,2),
    InterestRate DECIMAL(5,2),
    LoanTerm INT,
    ApplicationDate DATETIME,
    ApprovalDate DATETIME,
    LoanStatus VARCHAR(20),
```

```
    CONSTRAINT FK_Loan_Customer
    FOREIGN KEY (CustomerID)
    REFERENCES Customer(CustomerID)
);
```

**EMPLOYEE**

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    BranchID INT NOT NULL,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    JobTitle VARCHAR(50),
    Department VARCHAR(50),
    Email VARCHAR(50),
    PhoneNumber VARCHAR(15),
    EmploymentDate DATETIME,

    CONSTRAINT FK_Employee_Branch
    FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID)
);
```

**TRANSACTIONS**

```
CREATE TABLE Transactions (
    TransactionID INT PRIMARY KEY,
    AccountID INT NOT NULL,
    TransactionType VARCHAR(30),
    Amount DECIMAL(18,2),
    TransactionDate DATETIME,
    ReferenceNumber VARCHAR(50),
    Description VARCHAR(255),
    TransactionStatus VARCHAR(20),
```

```
    CONSTRAINT FK_Transactions_Account
    FOREIGN KEY (AccountID)
    REFERENCES Account(AccountID)
);
```

**CARD**

```
CREATE TABLE Card (
    CardID INT PRIMARY KEY,
    AccountID INT NOT NULL,
    CardNumber VARCHAR(20) UNIQUE,
    CardType VARCHAR(20),
    IssueDate DATETIME,
    ExpiryDate DATETIME,
    CardStatus VARCHAR(20),

    CONSTRAINT FK_Card_Account
    FOREIGN KEY (AccountID)
    REFERENCES Account(AccountID)
);
```

**TRANSACTION LOG**

```
CREATE TABLE Transaction_Log (
    LogID INT PRIMARY KEY,
    TransactionID INT NOT NULL,
    ActionPerformed VARCHAR(100),
    PerformedBy VARCHAR(100),
    LogTimestamp DATETIME,
    Remarks VARCHAR(255),

    CONSTRAINT FK_Log_Transaction
    FOREIGN KEY (TransactionID)
    REFERENCES Transactions(TransactionID)
);
GO
```

Database creation Proof.

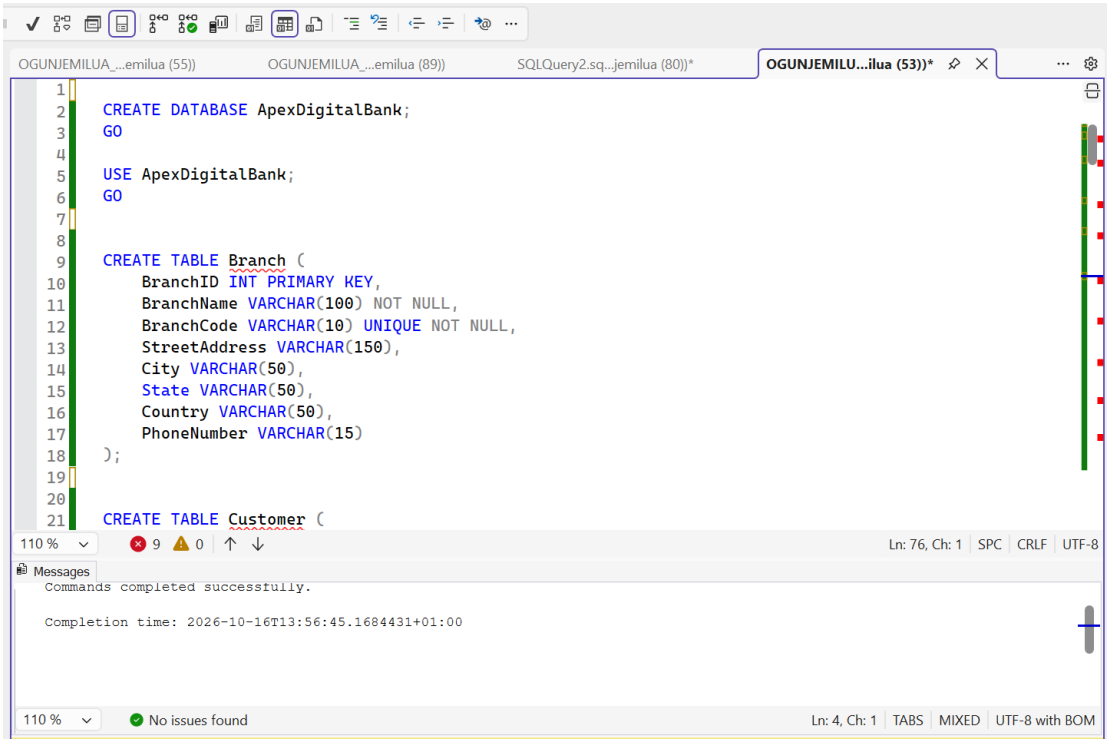
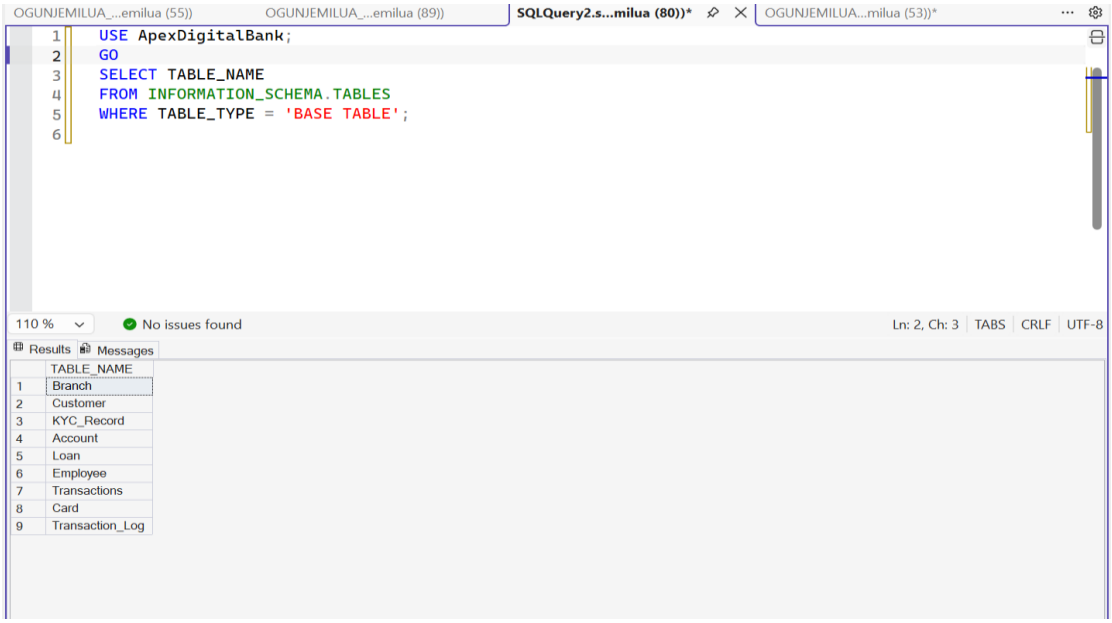



**Table Creation**

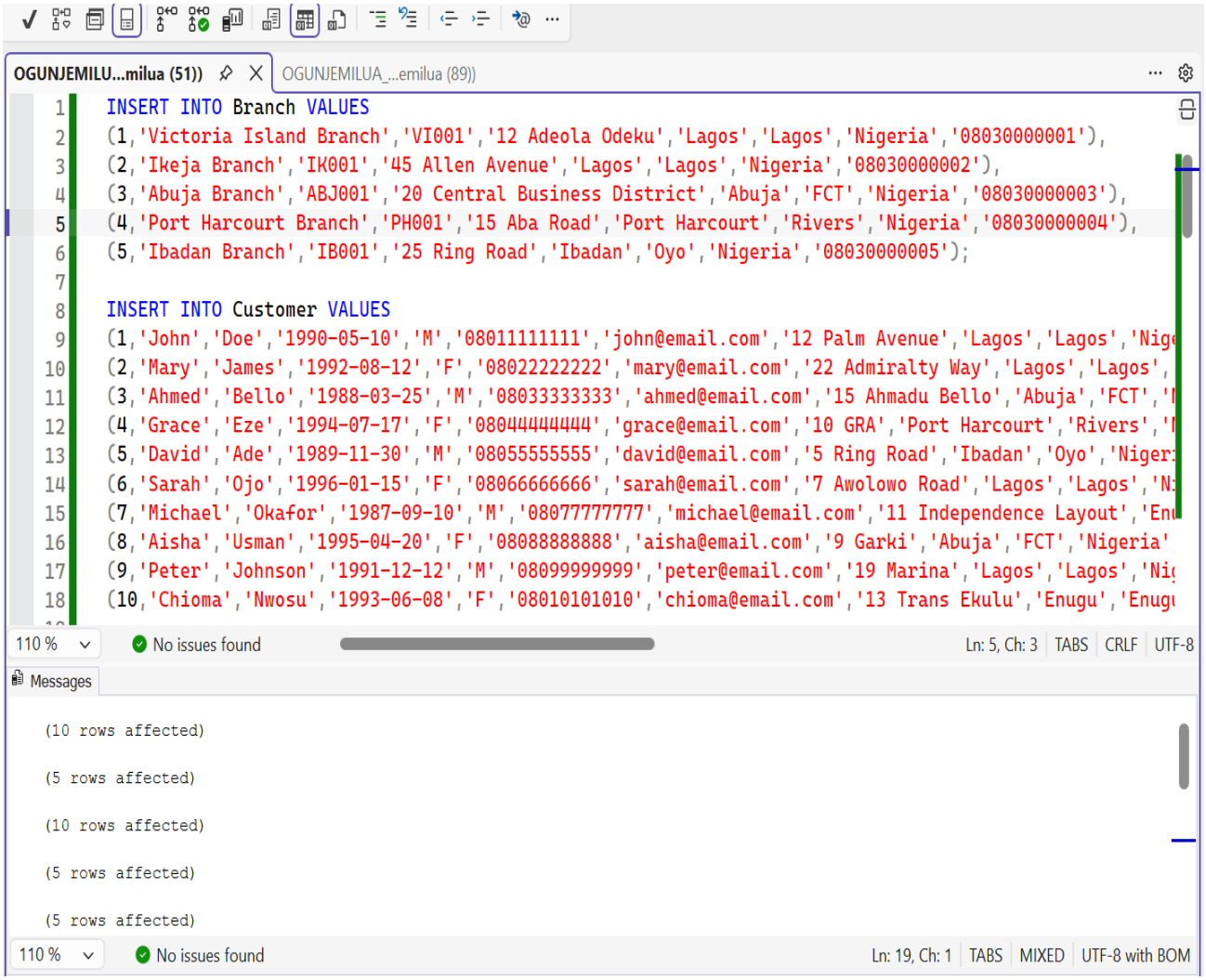
Screenshot showing successfully created tables.




## ➢ Data Population

Following the creation of the Apex Digital Bank database schema, sample data was inserted into all tables. To verify successful data population, SQL count queries were executed against each table. The results confirmed that records were successfully inserted across all entities, providing sufficient data for query processing, execution plan analysis, and performance optimization activities.

Database tables population.



```
INSERT INTO Branch VALUES
(1,'Victoria Island Branch','VI001','12 Adeola Odeku','Lagos','Lagos','Nigeria','08030000001'),
(2,'Ikeja Branch','IK001','45 Allen Avenue','Lagos','Lagos','Nigeria','08030000002'),
(3,'Abuja Branch','ABJ001','20 Central Business District','Abuja','FCT','Nigeria','08030000003'),
(4,'Port Harcourt Branch','PH001','15 Aba Road','Port Harcourt','Rivers','Nigeria','08030000004'),
(5,'Ibadan Branch','IB001','25 Ring Road','Ibadan','Oyo','Nigeria','08030000005');

INSERT INTO Customer VALUES
(1,'John','Doe','1990-05-10','M','08011111111','john@email.com','12 Palm Avenue','Lagos','Lagos','Nige
(2,'Mary','James','1992-08-12','F','08022222222','mary@email.com','22 Admiralty Way','Lagos','Lagos',
(3,'Ahmed','Bello','1988-03-25','M','08033333333','ahmed@email.com','15 Ahmadu Bello','Abuja','FCT','
(4,'Grace','Eze','1994-07-17','F','080444444444','grace@email.com','10 GRA','Port Harcourt','Rivers','
(5,'David','Ade','1989-11-30','M','08055555555','david@email.com','5 Ring Road','Ibadan','Oyo','Niger
(6,'Sarah','Ojo','1996-01-15','F','08066666666','sarah@email.com','7 Awolowo Road','Lagos','Lagos','N
(7,'Michael','Okafor','1987-09-10','M','08077777777','michael@email.com','11 Independence Layout','En
(8,'Aisha','Usman','1995-04-20','F','08088888888','aisha@email.com','9 Garki','Abuja','FCT','Nigeria'
(9,'Peter','Johnson','1991-12-12','M','08099999999','peter@email.com','19 Marina','Lagos','Lagos','Nig
(10,'Chioma','Nwosu','1993-06-08','F','08010101010','chioma@email.com','13 Trans Ekulu','Enugu','Enugu
```

```
(10 rows affected)

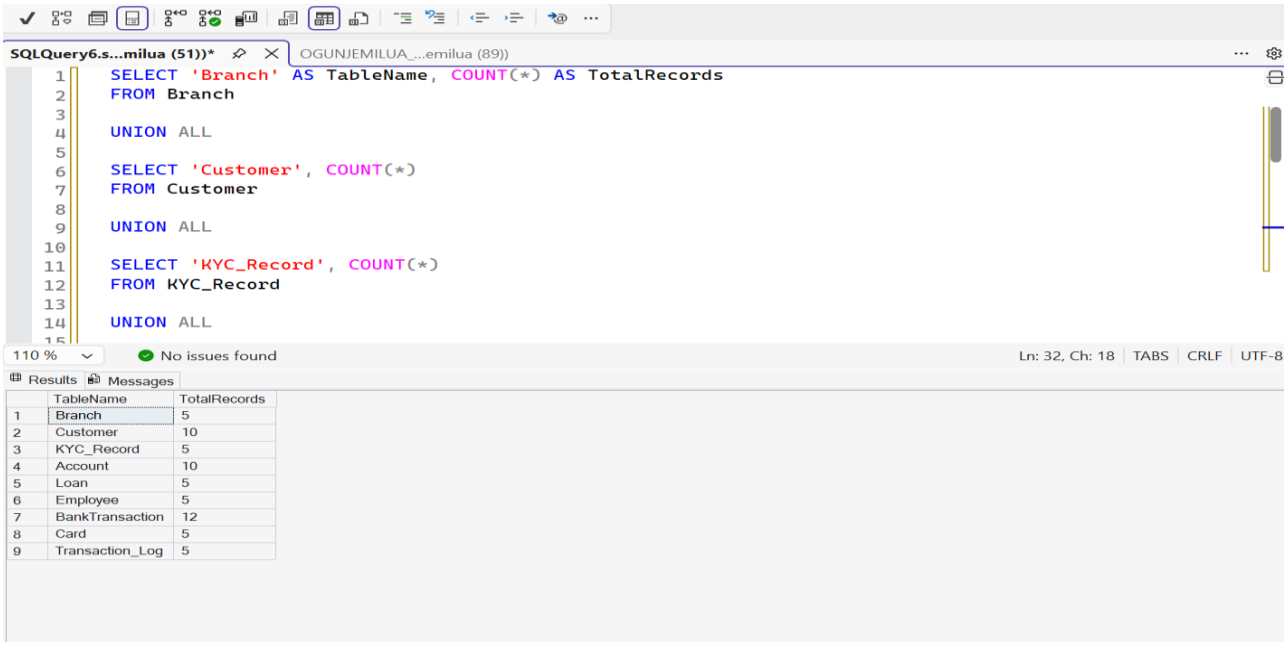
(5 rows affected)

(10 rows affected)

(5 rows affected)

(5 rows affected)
```

Evidence of table population.



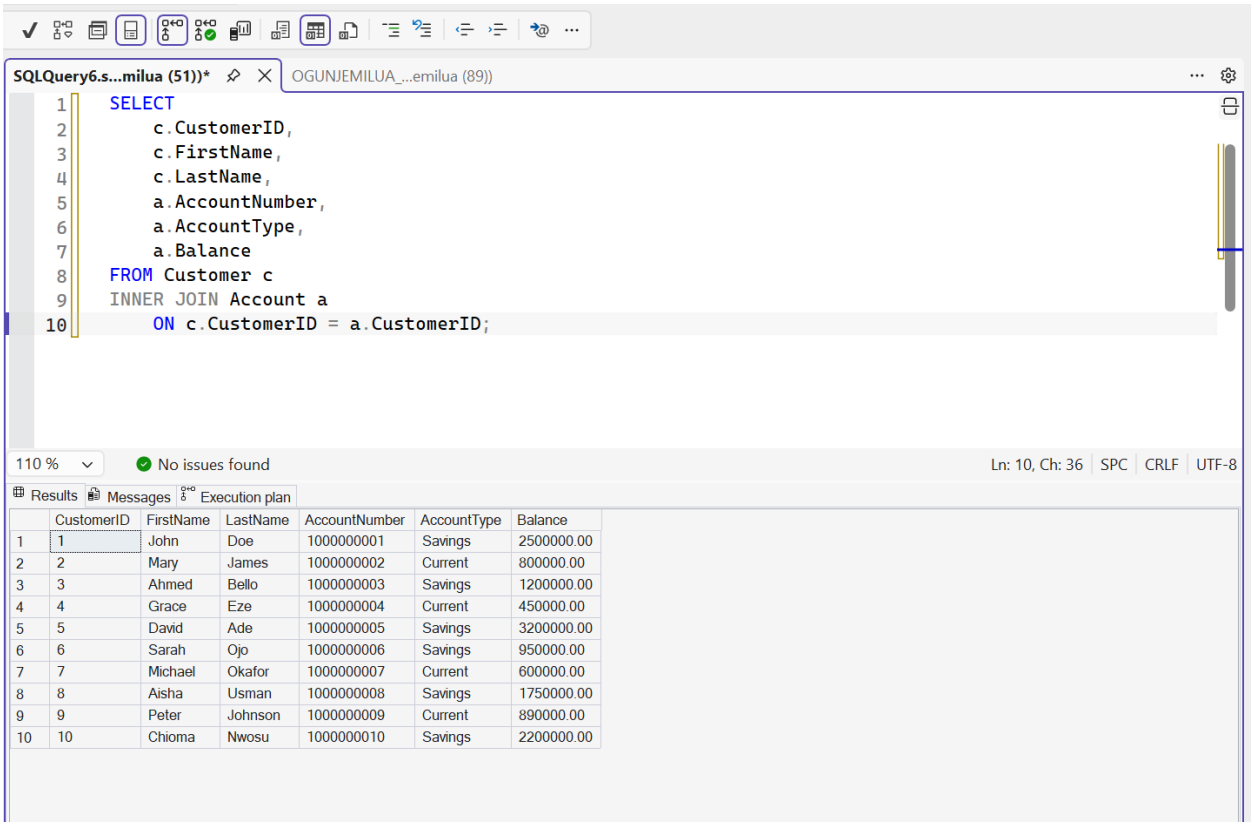

## ➢ Query Processing

This is the heart of Portfolio 2 as it shows us some transactions and query processing.

**Query 1: Customer Account Information -** This query retrieves customer records together with associated account information.

### SQL Statement

```
SELECT
   c.CustomerID,
   c.FirstName,
   c.LastName,
   a.AccountNumber,
   a.AccountType,
   a.Balance
FROM Customer c
INNER JOIN Account a
   ON c.CustomerID = a.CustomerID;
```

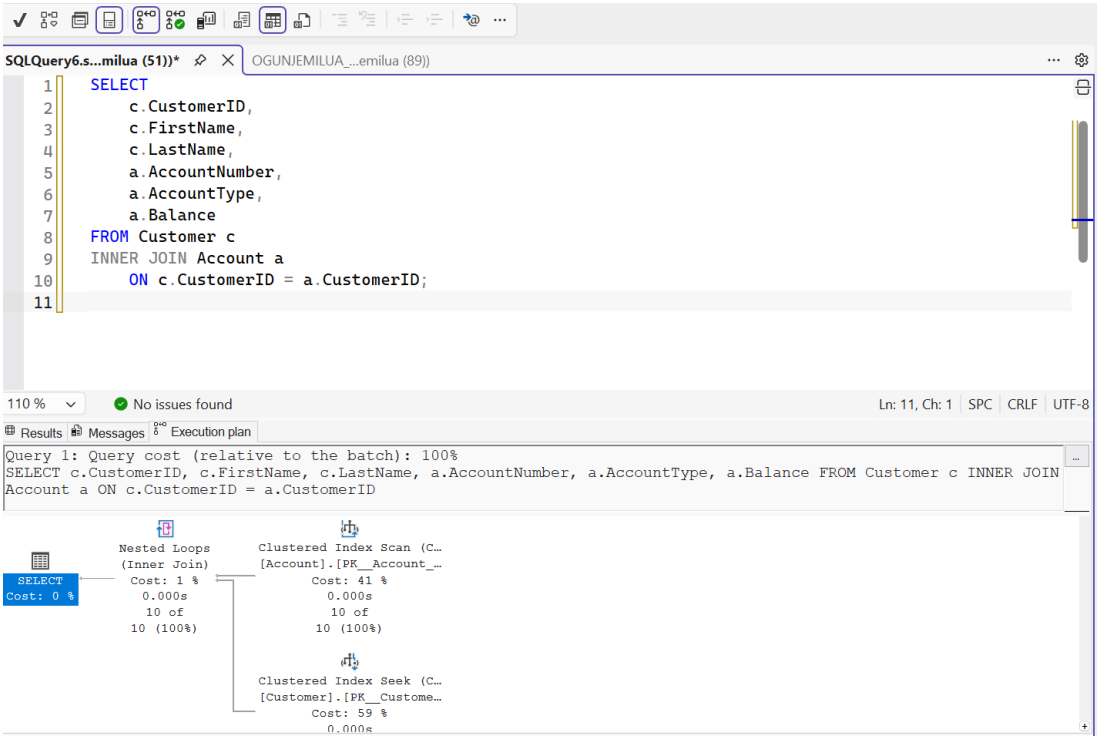



| | CustomerID | FirstName | LastName | AccountNumber | AccountType | Balance |
|---|---|---|---|---|---|---|
| 1 | 1 | John | Doe | 1000000001 | Savings | 2500000.00 |
| 2 | 2 | Mary | James | 1000000002 | Current | 800000.00 |
| 3 | 3 | Ahmed | Bello | 1000000003 | Savings | 1200000.00 |
| 4 | 4 | Grace | Eze | 1000000004 | Current | 450000.00 |
| 5 | 5 | David | Ade | 1000000005 | Savings | 3200000.00 |
| 6 | 6 | Sarah | Ojo | 1000000006 | Savings | 950000.00 |
| 7 | 7 | Michael | Okafor | 1000000007 | Current | 600000.00 |
| 8 | 8 | Aisha | Usman | 1000000008 | Savings | 1750000.00 |
| 9 | 9 | Peter | Johnson | 1000000009 | Current | 890000.00 |
| 10 | 10 | Chioma | Nwosu | 1000000010 | Savings | 2200000.00 |

**Analysis**

The execution plan shows that SQL Server joined the Customer and Account tables using the CustomerID attribute. The query successfully retrieved matching records from both tables and combined them into a single result set. This operation demonstrates how SQL Server processes join queries to retrieve related information efficiently.

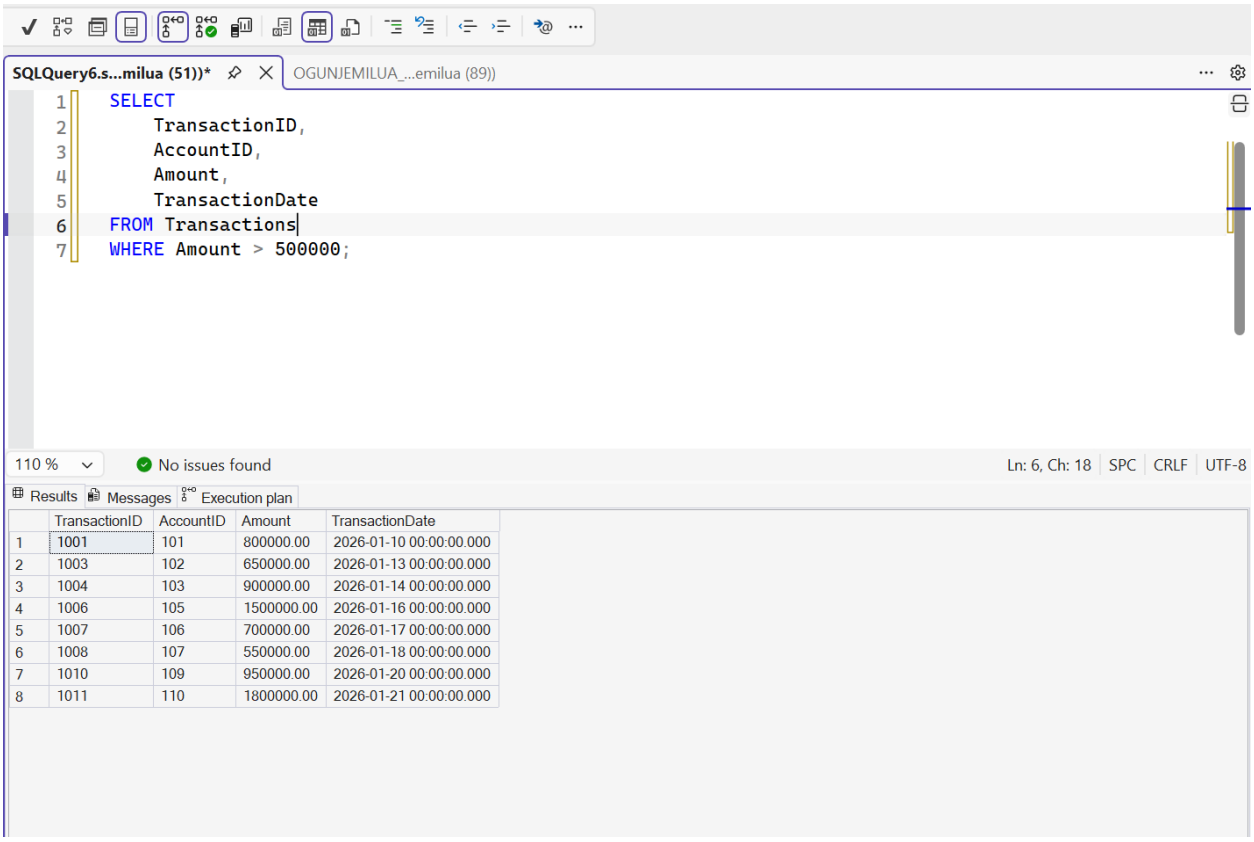
## Execution Plan




**Query 2: High-Value Transactions-** Identify large transactions for compliance, fraud detection and reporting.

## SQL Statement

```
SELECT
   TransactionID,
   AccountID,
   Amount,
   TransactionDate
FROM Transactions
WHERE Amount > 500000;
```



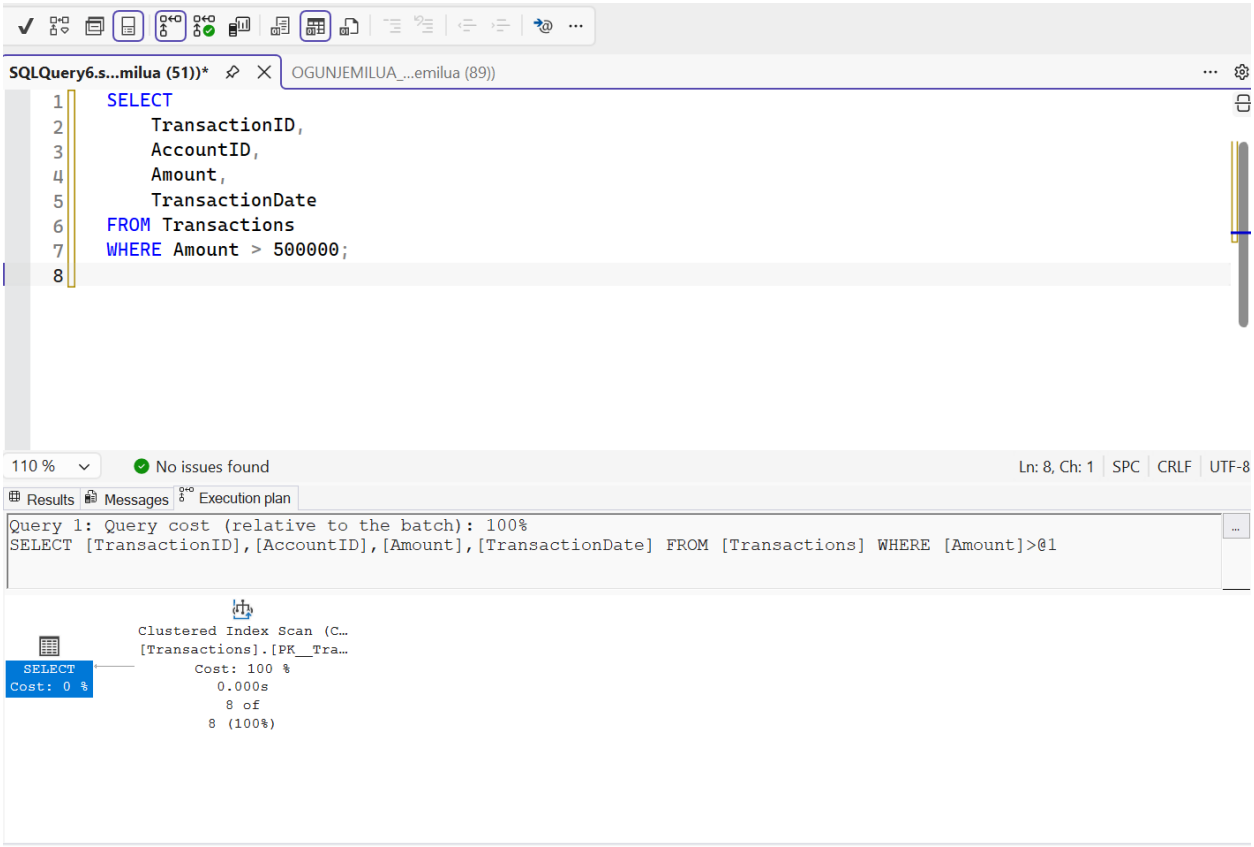

| | TransactionID | AccountID | Amount | TransactionDate |
|---|---|---|---|---|
| 1 | 1001 | 101 | 800000.00 | 2026-01-10 00:00:00.000 |
| 2 | 1003 | 102 | 650000.00 | 2026-01-13 00:00:00.000 |
| 3 | 1004 | 103 | 900000.00 | 2026-01-14 00:00:00.000 |
| 4 | 1006 | 105 | 1500000.00 | 2026-01-16 00:00:00.000 |
| 5 | 1007 | 106 | 700000.00 | 2026-01-17 00:00:00.000 |
| 6 | 1008 | 107 | 550000.00 | 2026-01-18 00:00:00.000 |
| 7 | 1010 | 109 | 950000.00 | 2026-01-20 00:00:00.000 |
| 8 | 1011 | 110 | 1800000.00 | 2026-01-21 00:00:00.000 |

**Analysis**

We used **where** clause to filter the data to get the exact data range we want.

The execution plan indicates that SQL Server applied a filtering condition to retrieve only transactions with amounts greater than ₦500,000. The filtering operation reduced the number of records returned and improved the relevance of the result set.

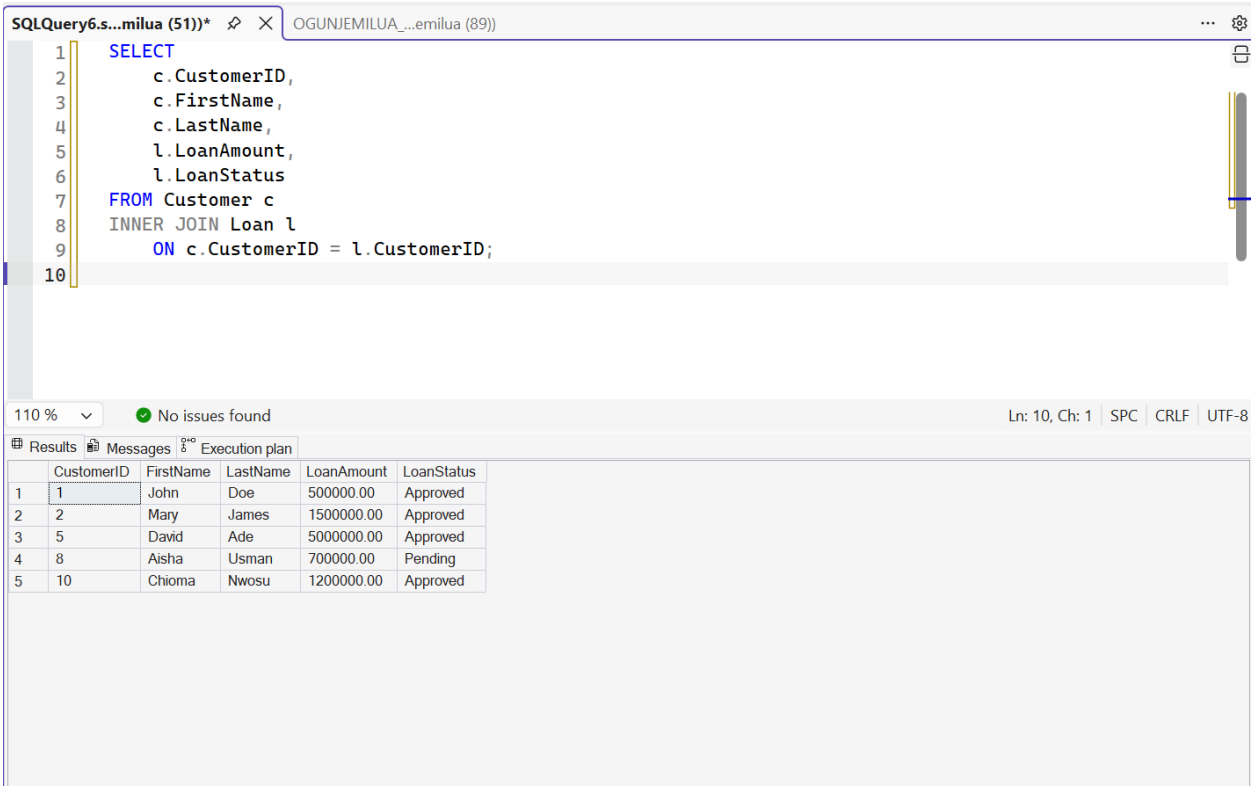
## Execution Plan




## Query 3: Customer Loans Information

## SQL Statement

```
SELECT
   c.CustomerID,
   c.FirstName,
   c.LastName,
   l.LoanAmount,
   l.LoanStatus
FROM Customer c
INNER JOIN Loan l
   ON c.CustomerID = l.CustomerID;
```

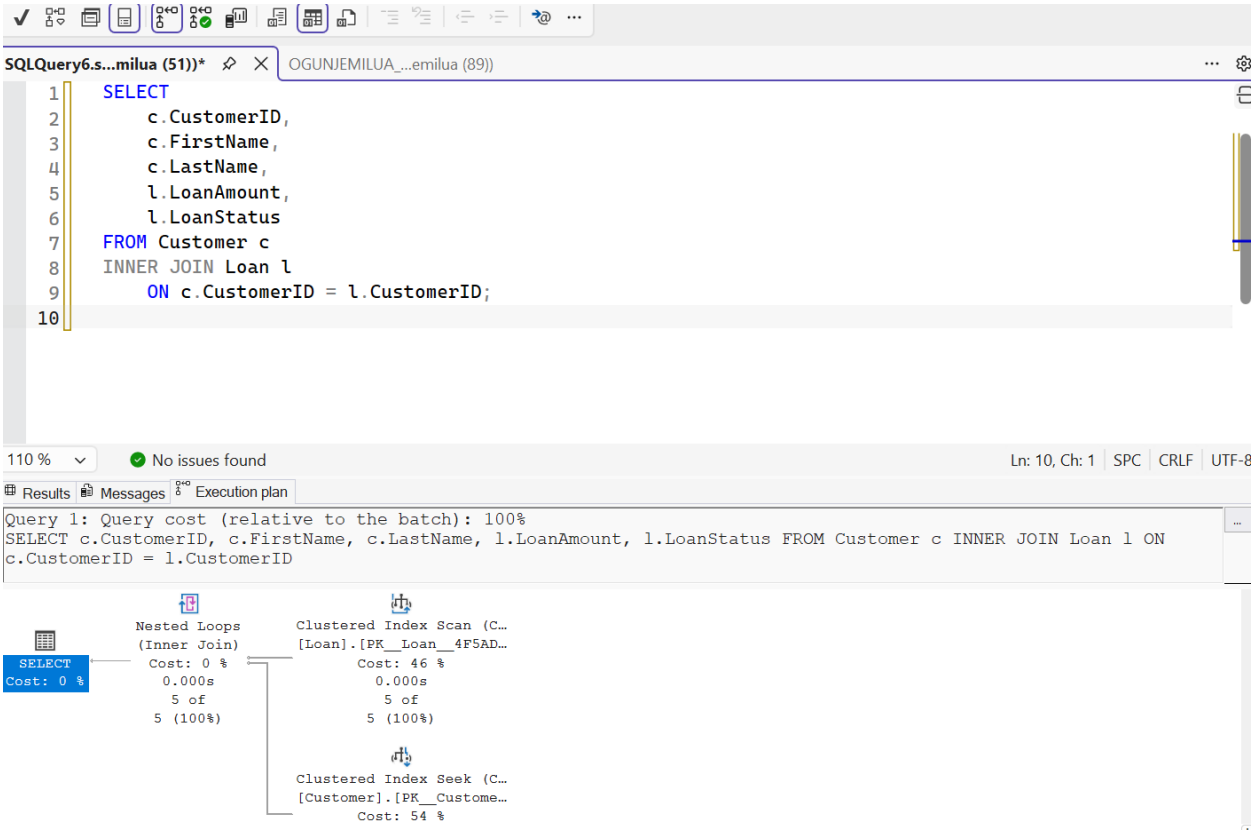



| | CustomerID | FirstName | LastName | LoanAmount | LoanStatus |
|---|---|---|---|---|---|
| 1 | 1 | John | Doe | 500000.00 | Approved |
| 2 | 2 | Mary | James | 1500000.00 | Approved |
| 3 | 5 | David | Ade | 5000000.00 | Approved |
| 4 | 8 | Aisha | Usman | 700000.00 | Pending |
| 5 | 10 | Chioma | Nwosu | 1200000.00 | Approved |

## Analysis

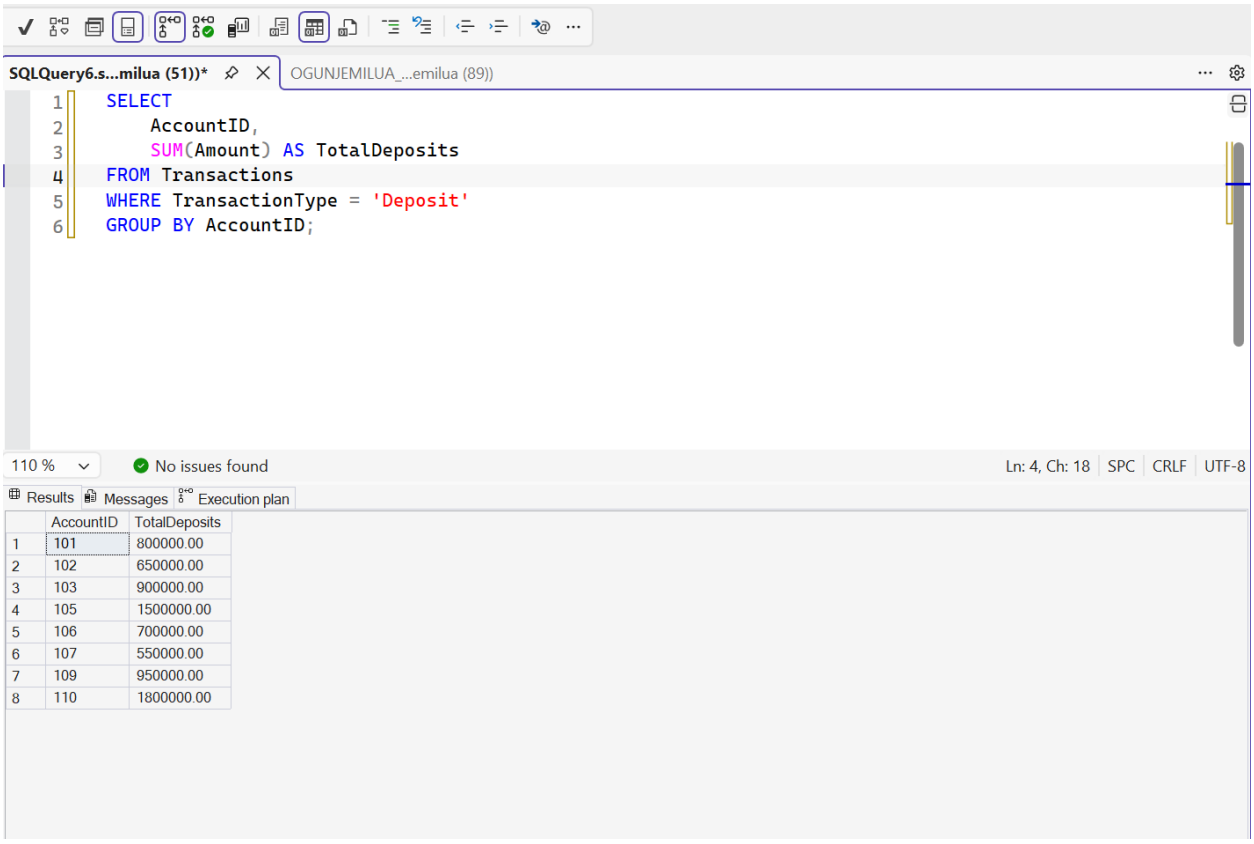
The execution plan shows a join operation between the Customer and Loan tables. SQL Server matched customer records with corresponding loan records through the CustomerID foreign key relationship.

## Execution Plan




## Query 4: Total Deposit Per Person – calculates total deposit made per person.

```
SELECT
   AccountID,
   SUM(Amount) AS TotalDeposits
FROM BankTransaction
WHERE TransactionType = 'Deposit'
GROUP BY AccountID;
```

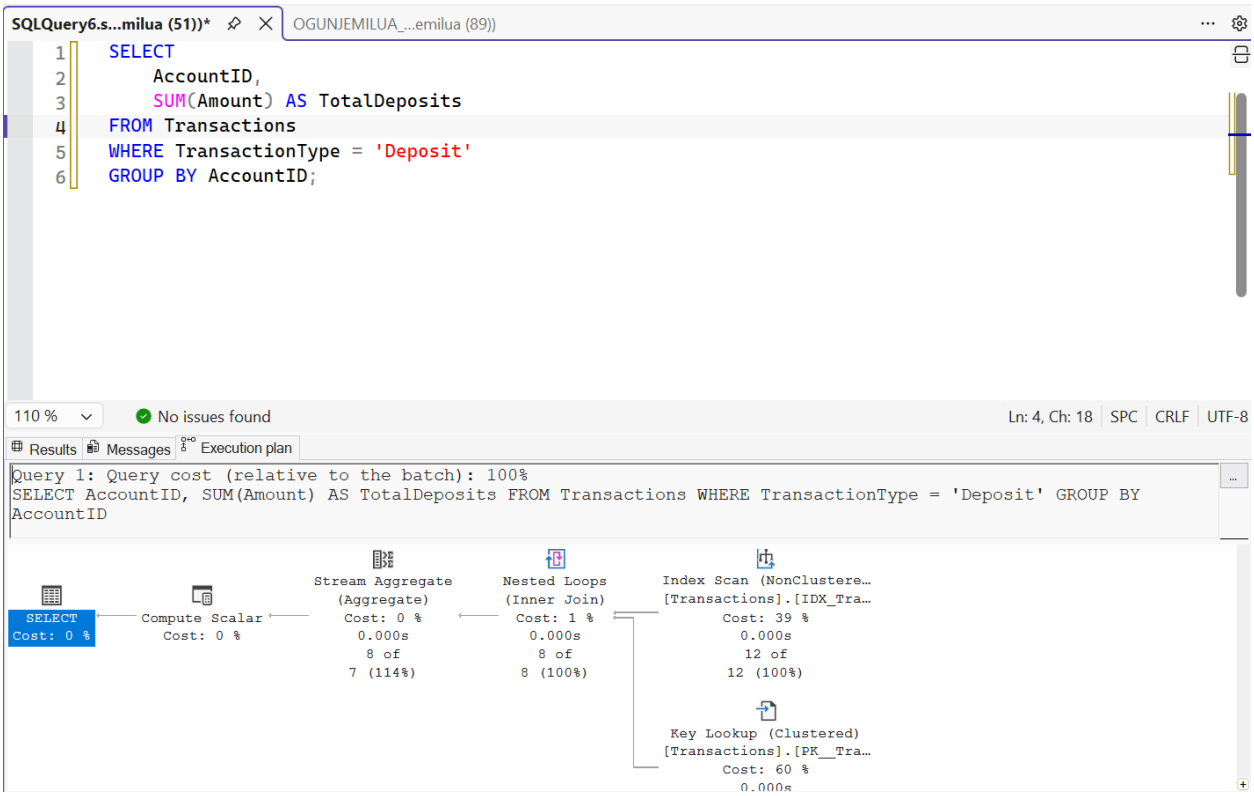



**Analysis**

We used aggregate function SUM here.

**Execution Plan Analysis**

The execution plan demonstrates the use of aggregation functions. SQL Server grouped records by AccountID and calculated the total deposit amount using the SUM() function.




## ➢ What is Execution Plan Explanation

The execution plan illustrates how SQL Server processed the query and accessed the relevant tables. The database engine selected an efficient execution strategy based on the query structure, available indexes, and table relationships. Operations such as filtering, joining, and data retrieval were performed to generate the final result set. The execution plan provides valuable insight into the query's resource usage and potential performance bottlenecks.

## ➢ Indexing and Optimization

### Index Creation

To improve query performance, indexes were created on columns frequently used in joins, filtering operations, and search conditions.

```
CREATE INDEX IDX_Account_Customer

ON Account(CustomerID);

CREATE INDEX IDX_Account_Branch

ON Account(BranchID);

CREATE INDEX IDX_Loan_Customer

ON Loan(CustomerID);
```

```
CREATE INDEX IDX_Transaction_Account
ON BankTransaction(AccountID);
CREATE INDEX IDX_Transaction_Amount
ON BankTransaction(Amount);
```

Indexes were implemented to improve data retrieval efficiency within the Apex Digital Bank database. The selected columns were chosen because they are commonly used in join conditions and query filters. By creating indexes on these attributes, SQL Server can locate records more efficiently, reducing the need for full table scans and improving query response times.

## ➢ Performance Analysis

Performance analysis was conducted to evaluate how SQL Server processed and executed the queries within the Apex Digital Bank database.

**Before Optimization**

Without indexes, SQL Server may perform full table scans to locate matching records. While this approach is acceptable for small datasets, it becomes increasingly inefficient as the database grows. Table scans consume additional CPU, memory, and I/O resources, leading to slower query execution.

**After Optimization**

The implementation of indexes enables SQL Server to locate records more efficiently using indexed access paths. This reduces the volume of data that must be examined during query execution. As a result, query execution becomes faster and more scalable, particularly for join operations and filtering conditions.

**Benefits of Indexing**

- Faster data retrieval
- Reduced execution time
- Lower disk I/O
- Improved query performance
- Better scalability for large datasets
- More efficient join operations

**Demerits of Indexing**

- Additional storage space required
- Slightly slower INSERT operations

- Slightly slower UPDATE operations
- Additional maintenance overhead

## ➢ Critical Evaluation

Query optimization is essential for maintaining database performance in modern banking environments. As transaction volumes increase, poorly designed queries can negatively affect system responsiveness and user experience. The use of indexes provides a practical mechanism for enhancing performance by reducing unnecessary data access operations. However, excessive indexing may introduce storage and maintenance costs. Therefore, database administrators must balance read performance improvements against the overhead incurred during data modification operations. The optimization approach adopted within the Apex Digital Bank database demonstrates how indexing can improve operational efficiency while maintaining data integrity and scalability.

## ➢ Conclusion

The Apex Digital Bank database was successfully implemented in Microsoft SQL Server. Queries were executed to support banking operations, execution plans were analyzed, and indexing strategies were applied to improve performance. The results demonstrated the importance of query optimization techniques in enhancing database efficiency, scalability and reliability.